AIAA SciTech Forum
11–15 & 19–21 January 2021, VIRTUAL EVENT
AIAA Scitech 2021 Forum

10.2514/6.2021-1837

Check for updates

A Matrix-free GMRES Algorithm on GPU Clusters for Implicit Large Eddy Simulation

Eduardo Jourdan* and Z. J. Wang†
Department of Aerospace Engineering, University of Kansas, Lawrence, KS, 66045, USA

This paper investigates the performance of the matrix-free GMRES algorithm without a preconditioner on GPU clusters for large eddy simulations (LES) with the high-order FR/CPR method. This implicit solution algorithm does not need to store the Jacobian matrix, which is often too large to store on GPUs. In addition to the residual operator, it is important to have an efficient global reduction operation. The present work considers various solution approximate orders on mixed unstructured meshes with four different element types: hexahedrons, prisms, pyramids and tetrahedrons. Issues related to their difference in speedup are discussed and compared with the explicit three-stage Runge-Kutta scheme on a GPU cluster. A strong scalability study is conducted on the Summit supercomputer with up to 1200 NVIDIA V100 GPUs. A LES benchmark case is used to evaluate different parameters and the convergence tolerance of the GMRES solver. Preliminary performance results are also described for a test case using the High Lift Common Research Model from the 4th AIAA High-Lift Prediction Workshop. In this case, a reduction of the simulation cost by a factor of 15 to 35 is obtained compared to an explicit scheme on a GPU cluster.

I. Introduction

A series of International Workshops on High-Order CFD Methods have been organized since 2012 [1] where benchmark vortex-dominated turbulent flow problems have been proposed and simulated using high-order methods. One major conclusion from them is that high-order methods are more efficient than low order ones for scale-resolving simulations such as large eddy simulation (LES) to achieve the same accuracy. Moreover, NASA's 2030 CFD vision states that LES will be increasingly utilized in order to handle separated flow problems such as flow over high lift configurations and through aircraft engines [2].

The use of high-order methods such as discontinuous Galerkin (DG) [3] and flux reconstruction (FR) [4] for LES has grown significantly in the last decade [5–10] because of the capability of these methods in handling complex geometries with unstructured meshes. Another advantage is the fact that they achieve high-order accuracy by increasing the number of degrees of freedom (DOFs) inside each element, remaining a compact scheme and hence improving its parallel performance. However, some challenges still remain, such as the need to further reduce the cost and turnaround time of these simulations.

At the same time, modern hardware is shifting towards the use of highly parallel accelerators, like for instance NVIDIA's Graphical Processing Units (GPUs). Several world flagship supercomputers such as Summit (Oak Ridge National Laboratory), Sierra (Lawrence Livermore National Laboratory) and Piz Daint (Swiss National Supercomputing Centre) are equipped with GPU-powered nodes. GPUs are capable of achieving a much higher theoretical peak in floating-point operations per second (FLOPs) by using a different parallelism strategy, executing concurrently the same instruction for a large set of data. As of October 2020, 6 of the top 10 super-computers on the Top500 list [11] have a node configuration with GPUs. Most recent GPU developments include the launch of the Nvidia A100 and AMD MI100 cards with increasingly higher memory bandwidth, memory resources and computational power.

Typical challenges in the use of GPU for high-order CFD simulations involve [10, 12–20] different parallel architecture, data-communication and scalability issues, memory access pattern and memory bandwidth limitation. Nevertheless, high-order methods such as DG and FR are compact with intensive local operations, with the DOFs inside an element increasing rapidly with the the order of accuracy. This leads to operations like polynomial interpolations and matrix vector multiplications being faster and more efficient for this particular hardware architecture, making these methods a suitable choice to be used on GPUs.

*PhD Student, Department of Aerospace Engineering; 2120 Learned Hall; E-mail: ejourdan@ku.edu.
†Spahr Professor, Department of Aerospace Engineering; E-mail: zjw@ku.edu. Fellow AIAA.

Copyright © 2021 by Eduardo Jourdan and Z. J. Wang . Published by the American Institute of Aeronautics and Astronautics, Inc., with permission.

Downloaded by Politecnico di Milano on February 4, 2026 | http://arc.aiaa.org | DOI: 10.2514/6.2021-1837

One the other hand, higher-order spatial operators are much stiffer than low order ones [21]. As a result, the stability limit of explicit schemes is more severe for high-order methods, and matrix-based implicit methods take much more memory than lower order ones [21]. The different memory architecture of the GPU creates additional difficulties for the use of implicit time solvers with high-order methods [18]. Implicit solvers also tend to require more communication and may have memory bound algorithms like global reduction operations.

A typical implicit solver used in CFD is the Newton-Krylov method, in which Newton-type methods for the nonlinear system are coupled with Krylov subspace methods for linear system solving [22]. Reference [18] shows some of the difficulties in accelerating implicit solvers like the GMRES method [23] with matrix-based preconditioners like the inherently sequential incomplete lower-upper factorization (ILU). The speedups between one Nvidia K40 GPU and one CPU core for explicit schemes are on the order of 70, while it is around 5 for a GMRES+ILU method [18]. Some additional works with acceleration techniques include Ref. [14] with a preconditioned conjugate gradient solver for the Poisson equation and Ref. [10] with a p-multigrid method and explicit smoothers.

In this context, the objective of the current paper is to investigate the use of a matrix-free implementation of the GMRES method on multi-GPU clusters for unsteady simulations. The matrix-free GMRES method [22] has a much smaller memory footprint, and besides the residual operator, it needs an efficient global reduction algorithm and internal product calculation. Since the focus of the current work is on unsteady simulations, no preconditioner is used because this system of equations has a better condition number than steady problems since a small time step is needed to capture the turbulent dynamics.

First, the speedup between CPU and GPU implementations of the matrix-free GMRES method is measured for several orders of accuracy and cell types. Different parts of the algorithm such as the residual operator and the Gram-Schmidt orthogonalization process and their performance are investigated. Strong scalability studies from p1 to p5 are done with up to 200 nodes and 1200 GPUs on the Summit supercomputer, which is the number two computer in the TOP500 list of June 2020 [11].

The LES benchmark T106C turbine case is used to run parametric tests involving the nonlinear unsteady residual convergence tolerance, the GMRES linear convergence tolerance and the Krylov subspace size. The simulation accuracy and total turnaround time is compared between a multi-GPU GMRES implementation, an explicit RK3 time scheme run on multi-GPUs and an implicit LU-SGS solver on a CPU cluster. Finally, the GPU GMRES algorithm performance is tested on a mixed mesh of the high-lift configuration version of the Common Research Model intended to be studied on the 4th AIAA CFD High Lift Prediction Workshop.

The paper is organized as follows. Section II presents the numerical formulation of the high-order FR/CPR method and section III describes the matrix-free GMRES algorithm. Finally, Sec. IV presents the numerical results and Sec. V draws some concluding remarks.

II. A Brief Overview of the FR/CPR Method

The FR/CPR method was originally developed by Huynh [4] in 2007 for hyperbolic partial differential equations, and later it was extended to hybrid unstructured meshes [24]. Further developments on the FR/CPR method are reviewed in [25, 26]. This method belongs to discontinuous finite element methods, similar to the DG method, but also has some unique advantages. For example, FR contains a larger family of schemes [4], which may allow larger time steps than the DG method [27]. Many groups also reported that FR is more efficient than the DG method [28, 29]. Here we present a brief introduction of the FR/CPR method starting from a hyperbolic conservation law

$$\frac{\partial \boldsymbol{U}}{\partial t} + \nabla . \boldsymbol{F}(\boldsymbol{U}) = 0, \tag{1}$$

with initial and boundary conditions, where the vector $\boldsymbol{U}$ consists of conservative variables, and $\boldsymbol{F}$ is the flux. By discretizing the computational domain with non-overlapping elements, and introducing an arbitrary test function W in each element, the weighted residual formulation of Eq. 1 on element V_i can be expressed as

$$\int_{V_i} \left[\frac{\partial \boldsymbol{U}}{\partial t} + \nabla . \boldsymbol{F}(\boldsymbol{U}) \right] W d\Omega = 0, \tag{2}$$

The conservative variables inside one element are assumed to be polynomials, and expressed by nodal values at certain points called solution points (SPs). After applying integration by parts to the divergence of flux, replacing the normal flux term with a common Riemann flux F^n_{com} and integrating back by parts, we obtain

$$\int_{V_i} \frac{\partial \boldsymbol{U_i}}{\partial t} W d\Omega + \int_{V_i} W \nabla . \boldsymbol{F}(\boldsymbol{U_i}) d\Omega + \int_{\partial V_i} W \left[F^n_{com} - F^n(\boldsymbol{U_i}) \right] dS = 0. \tag{3}$$

Downloaded by Politecnico di Milano on February 4, 2026 | http://arc.aiaa.org | DOI: 10.2514/6.2021-1837

Here, the common Riemann flux is computed with a Riemann solver

$$F^n_{com} = F^n_{com}(\boldsymbol{U_i}, \boldsymbol{U_{i+}}, \boldsymbol{n}), \tag{4}$$

where U_{i+} stands for the solution outside the current element, and $\boldsymbol{n}$ denotes the outward normal direction of the interface. For the current work, the Riemann solver used is the Roe scheme [30]. The normal flux at the interface is:

$$F^n(\boldsymbol{U_i}) = \boldsymbol{F}(\boldsymbol{U_i}) \cdot \boldsymbol{n}. \tag{5}$$

Note that if the face integral in Eq. 3 can be transformed into an element integral then the test function W will be eliminated. In order to do so, a "correction field" δ_i is is defined in each element as

$$\int_{\partial V_i} W \delta_i d\Omega = \int_{\partial V_i} W[F^n] dS, \tag{6}$$

where $[F^n] = \left[F^n_{com} - F^n(\boldsymbol{U_i})\right]$ is the normal flux jump. Eqs. 3 and 6 result in

$$\int_{V_i} \left[\frac{\partial \boldsymbol{U_i}}{\partial t} + \nabla . \boldsymbol{F}(\boldsymbol{U_i}) + \delta_i \right] W d\Omega = 0. \tag{7}$$

The final formulation for each solution point i is

$$\frac{\partial \boldsymbol{U_{i,j}}}{\partial t} + \prod_j [\nabla . \boldsymbol{F}(\boldsymbol{U_i})] + \delta_{i,j} = 0, \tag{8}$$

where $\prod_j$ denotes a projection to the polynomial space, and subscript j denotes the j-th solution point in a certain element. The formulation used for the viscous flux and the solution gradient is the so called BR2 scheme [31].

III. Matrix-free GMRES Algorithm

The GMRES algorithm developed by Saad and Schultz [23] is an iterative linear system solver. It is widely used in the CFD community, including with high-order methods [21]. However, its efficiency is highly influenced by the matrix condition number [22] and typically high-order discretizations lead to stiffer problems. Furthermore, the Jacobian matrix size derived from high-order methods grows very quickly with the solution polynomial order p. It is proportional to p^6 for 3-D simulations.

The matrix free version of the GMRES algorithm takes advantage of the fact that the Jacobian matrix is used only on matrix-vector multiplications. Therefore, one do not need to form the matrix exactly. Different approximations can be used for this operation [22], and in this work a simple first order finite difference approach is used like in Eq. 13.

A robust and efficient preconditioner that scales well on massive parallel systems is still a research challenge [21]. This current work does not use any preconditioner. The fact that only unsteady simulations are performed helps to partially mitigate this issue. Previous works have shown the importance of preconditioners for steady simulations starting the iterative process from the freestream solution [21, 22, 32].

The framework used here is often called the Newton-Krylov method [22]. It has an outer non-linear solver loop like the Newton method that is used to drive the unsteady residual to zero and an inside iterative linear system solver like the GMRES method to solve the linear system that originates from the Newton method. The following is the derivation of this framework.

Assume that after the spatial discretization the semi-discrete problem can be written as

$$\frac{d\hat{\boldsymbol{U}}}{dt} = \boldsymbol{R}(\hat{\boldsymbol{U}}), \tag{9}$$

in which $\hat{\boldsymbol{U}}$ represents all the degrees of freedom in the problem and $\boldsymbol{R}$ is the residual operator given by the spatial discretization. The problem can be rewritten so that in every physical time step the following nonlinear system is solved:

$$\boldsymbol{R}^*(\hat{\boldsymbol{U}}) = \boldsymbol{R}(\hat{\boldsymbol{U}}) - \frac{d\hat{\boldsymbol{U}}}{dt} = 0, \tag{10}$$

Downloaded by Politecnico di Milano on February 4, 2026 | http://arc.aiaa.org | DOI: 10.2514/6.2021-1837

where $\boldsymbol{R}^*$ is the unsteady residual. The second order backward differentiation formula (BDF2) time scheme is used for the time derivative discretization in this work.

A Newton solver can be used to solve this system, with the Jacobian matrix J defined as

$$\boldsymbol{J} = \frac{\partial \boldsymbol{R}^*(\hat{\boldsymbol{U}})}{\partial \hat{\boldsymbol{U}}}. \tag{11}$$

The Newton method consists of a sequence of iterations k with the following linear systems to be solved:

$$\boldsymbol{J}(\hat{\boldsymbol{U}}^k)\delta\hat{\boldsymbol{U}}^k = -\boldsymbol{R}^*(\hat{\boldsymbol{U}}^k)\ , \hat{\boldsymbol{U}}^{k+1} = \hat{\boldsymbol{U}}^k + \delta\hat{\boldsymbol{U}}^k\ , k = 0, 1, \dots\ , \tag{12}$$

with a given initial condition $\hat{\boldsymbol{U}}^0$ equal to the current time step solution. The Newton iteration can be terminated based on a required drop in the norm of the unsteady nonlinear residual defined in Eq. 10. This nonlinear convergence tolerance here named 'time-eps' can affect the final solution accuracy, and its value is studied in this paper.

In order to solve the linear system defined in Eq. 12 we use the matrix-free GMRES method without a preconditioner. This linear system can be solved up to some tolerance named 'linear-eps', and this parameter's effect on the method efficiency and convergence is studied in this paper too. The matrix vector multiplication in the GMRES solver is replaced by the following approximation:

$$\boldsymbol{J}\boldsymbol{v} \approx [\boldsymbol{R}^*(\hat{\boldsymbol{U}}^k + \epsilon\boldsymbol{v}) - \boldsymbol{R}^*(\hat{\boldsymbol{U}}^k)]/\epsilon, \tag{13}$$

where ϵ is a small perturbation, here chosen to be $\epsilon = |\hat{\boldsymbol{U}}^k|10^{-7}$.

The pseudocode of the GMRES method for solving the problem $Jx = b$ is shown in Algorithm 1 following the reference [33]. The size of the Krylov subspace is given by the parameter $m \sim 15$, which is very small compared to the mesh size.

Algorithm 1 Pseudocode of the matrix-free GMRES algorithm for solving $Jx = b$

1: Compute $r_0 = b - Jx_0$, $\beta = |r_0|$ and $v_1 = r_0/\beta$.
2: **for** $l = 1, 2, ..., m$ **do**
3: Compute $w_l = Jv_l$ using Eq. 13.
4: Compute $h_{e,l}$ = DotProduct(w_l,v_e) for $e = 1, ..., l$.
5: Compute $w_l = w_l - \sum_{e=1}^{l} h_{e,l}v_e$
6: $h_{l+1,l} = |w_l|$
7: $v_{l+1} = w_l/h_{l+1,l}$
8: **end for**
9: Define the $(m + 1) \times m$ matrix $H_m = [h_{e,l}]_{1\le e\le m+1, 1\le j\le m}$
10: Compute the solution y_m of the least square problem $|\beta e - H_m y|$.
11: Compute $x_m = x_0 + V_m y_m$, where $V_m = [v_1, v_2, ..., v_m]$

It is important to draw attention to some points in this algorithm. First, lines 4 and 5 are the classical Gram Schmidt (CGS) orthogonalization algorithm. Typically the modified Gram Schmidt process could be used here to provide robustness for ill-conditioned problems [33], but the CGS algorithm is chosen since it is more suitable for GPU parallelism. No impact in the robustness and stability of the algorithm was noticed by using the classical method instead of the modified one in the cases considered in the current work. The CGS algorithm has a better performance on GPUs because it involves matrix-vector operations instead of just vector-vector operations in the modified algorithm.

Secondly, note that the cost of line 3 is basically one residual evaluation. Therefore, it is important to have an efficient implementation of the residual calculation [19]. The numerical results in Sec. IV show that most of the runtime is spent on the residual calculation in line 3 and in the CGS algorithm.

Another important aspect of the GMRES solver is that an efficient global reduction and dot product kernel on GPU is important for lines 1, 4 and 6 in the Algorithm 1. The current implementation uses the warp shuffle reduction proposed by Nvidia to reduce the local vector inside the GPU to a scalar, and then we use a CUDA aware MPI implementation to obtain the final results across multi-GPUs.

The least square problem defined in line 10 has a very small size, of the order of the number m of Krylov vectors used. Moreover, in the current work a common technique is used to solve it, that is, to transform the Hessenberg

Downloaded by Politecnico di Milano on February 4, 2026 | http://arc.aiaa.org | DOI: 10.2514/6.2021-1837

matrix into upper triangular form by using plane rotations [33]. This technique allows to check the convergence of the method in each iteration l without extra work, and the final solution update y_m is calculated on the CPU by a backward substitution algorithm. In addition, note that if the linear system is not solved up to the tolerance requested, the current implementation uses the GMRES output to update the solution anyway and moves on to the next non-linear iteration. The restarted version of the GMRES is not used to continue to find the linear system solution.

Following this description of the Newton-Krylov approach, there are three major parameters: the unsteady residual convergence tolerance 'time-eps', the linear system convergence tolerance 'linear-eps' and the maximum size m of the search space. Their impact are numerically investigated in the following section. The parameter 'time-eps' is related to the accuracy of the time discretization, and both 'linear-eps' and m are related to the performance of the GMRES.

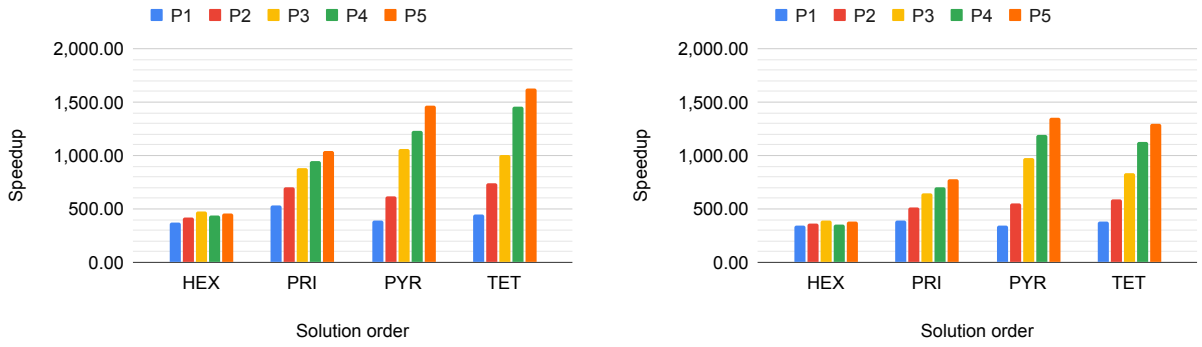
In addition, two other methods are used for performance comparison: the LU-SGS [34, 35] solver with BDF2 time scheme and the third order explicit strong stability-preserving Runge-Kutta scheme (RK3) [36]. The LU-SGS solver is implemented only for CPUs and its performance for the current code is shown in Ref. [8]. The RK3 scheme is also implemented on GPUs, with its performance described in Ref. [8, 19].

Downloaded by Politecnico di Milano on February 4, 2026 | http://arc.aiaa.org | DOI: 10.2514/6.2021-1837

IV. Numerical Results

A. Multi-GPU Performance and Scalability Results

In this section, results related to single and multi-GPU performance are described and discussed. The goal is to investigate the difference in performance between simulations with four cell types and different solution orders. Fig. 1 shows the speedup obtained with respect to a single CPU core of an Intel Xeon CPU E5-2620 chip. This Intel chip has 6 cores and hyper-threading capability, but the comparison here is with respect to just a single core performance.



(a) Explicit RK3 scheme.

(b) GMRES solver with BDF2 time scheme.

Fig. 1 NVIDIA V100 GPU Speedup compared to a single CPU core of an Intel Xeon CPU E5-2620 chip.

The GPU speedup for the explicit scheme with hexahedral cells is around 400 for all orders considered. It increases slightly for p3 and p5, when it becomes 475 and 455 respectively. On the other hand, the speedup results for tetrahedral cells increase with the solution polynomial order. It grows from 450 to 1600 for p1 and p5 respectively. This is discussed by Ref. [19], and one of the reasons is that a tetrahedral cell is more arithmetic intense and it has a much bigger ratio between floating point operations and global memory access as the polynomial degree increases [19]. As also discussed in Ref. [19], a pyramid cell follows closely the tetrahedron performance and the prism speedup is between a hexahedron and a tetrahedron.

The GMRES results in Fig. 1 show a similar trend between different cell types as the explicit scheme, but with a slightly lower speedup. For hexahedral cells, it is 390 and 380 for p3 and p5 respectively, while for tetrahedral cells it is 380 and 1300 for p1 and p5 respectively. This slightly lower values can be understood by the fact that almost all of cost of the explicit iteration comes from the residual operator, which is very arithmetic intense as discussed before [19] and involves matrix-vector and matrix-matrix operations.

On the other hand, the GMRES solver has additional operations other than the residual calculation. As described in Sec. III, they are mainly vector-vector operations and global reductions, which have a lower parallel speedup in the GPU architecture than matrix-matrix operations. Table 1 shows the percentage measured of a typical simulation iteration that

is spent in the residual operator (line 3 of Algorithm 1) and in the classical Gram-Schmidt (CGS) algorithm (line 4 and 5 of Algorithm 1). We can see that the CGS method account for 7% to 18% of the iteration cost. Interestingly, this

Table 1 Results of the percentage of a typical simulation iteration that is spent in different parts of the GMRES algorithm. Results for different solution order and cell types.

	HEX		PRI		PYR		TET	
Solution order	Residual	CGS	Residual	CGS	Residual	CGS	Residual	CGS
p1	75%	11%	74%	12%	74%	13%	78%	10%
p3	67%	15%	71%	12%	73%	11%	75%	10%
p5	64%	18%	69%	15%	79%	7%	76%	10%

percentage increases with the polynomial order for the hex and prism elements, which have considerably higher number of degrees per cell than the pyramid and tet elements for higher order.

Another important aspect investigated is the strong scalability performance of the matrix-free GMRES solver. Strong scalability tests are done with two different grids: a coarse hexahedral mesh with 149k cells and a fine hexahedral mesh with 15.6M elements. The coarse mesh is used on the KU cluster. In this test, 14 nodes are used, and each node has 2 V100 GPUs. Each node also has one Intel(R) Xeon(R) CPU E5-2680 v3 chip with 24 CPU cores after hyper threading. The network connection uses InfiniBand but the CUDA-aware MPI is not available. The results are shown in Fig. 2.

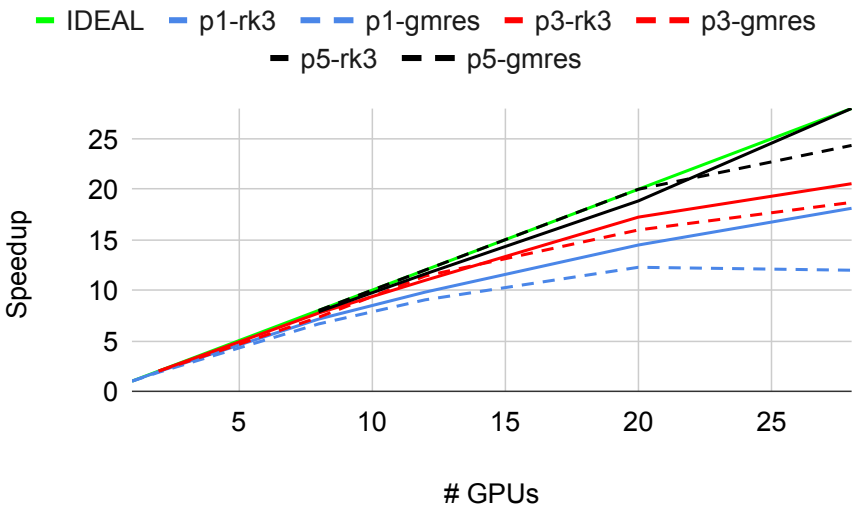


Fig. 2 Strong scalability study with GPU simulations on the KU cluster for a mesh with 149k hex elements.

We can see that the parallel efficiency grows with the polynomial order. Moreover, the difference between the scalability of the explicit scheme and the GMRES solver is not so significant for p3 and p5 as it is for p1. With 28 GPUs the number of cells per partition is 5.3k and the GMRES parallel efficiency is 43%, 67% and 87% for p1, p3 and p5 respectively.

The strong scalability results for the fine hex mesh on Summit are shown in Fig. 3. The Summit supercomputer is a leadership computing facility operated by the Oak Ridge National Laboratory and the United States Department of Energy. From June of 2018 to 2020, Summit was the the most powerful supercomputer in the TOP500 list [11]. Each node on Summit has 6 V100 GPUs and 2 IBM POWER9 CPU chips totalling 44 CPU cores. Here, CUDA aware MPI communication is used.

The results in Fig. 3 indicate again that the GMRES parallel efficiency grows with the solution order. Other than the p1 case, the scalability of the explicit scheme and the GMRES solver seem similar. More than 88% of parallel efficiency is achieved with 240 GPUs on all simulations considered. With 1200 GPUs, the number of cells per partition is 13k and the GMRES parallel efficiency is 21%, 45% and 79% for p1, p3 and p5 respectively.

Downloaded by Politecnico di Milano on February 4, 2026 | http://arc.aiaa.org | DOI: 10.2514/6.2021-1837

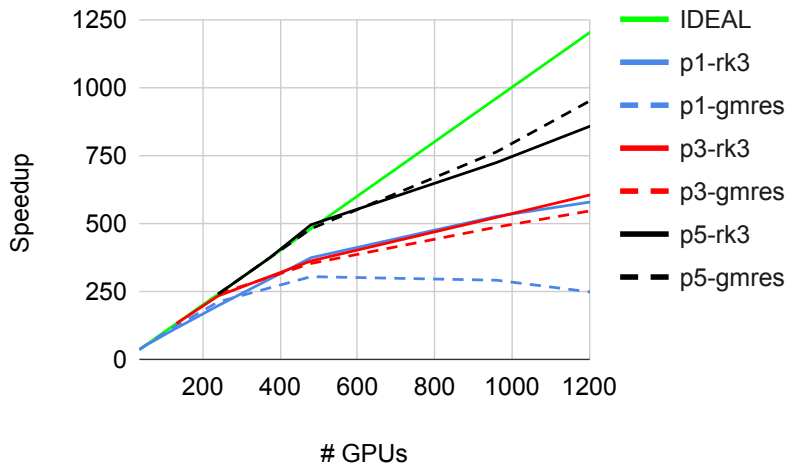


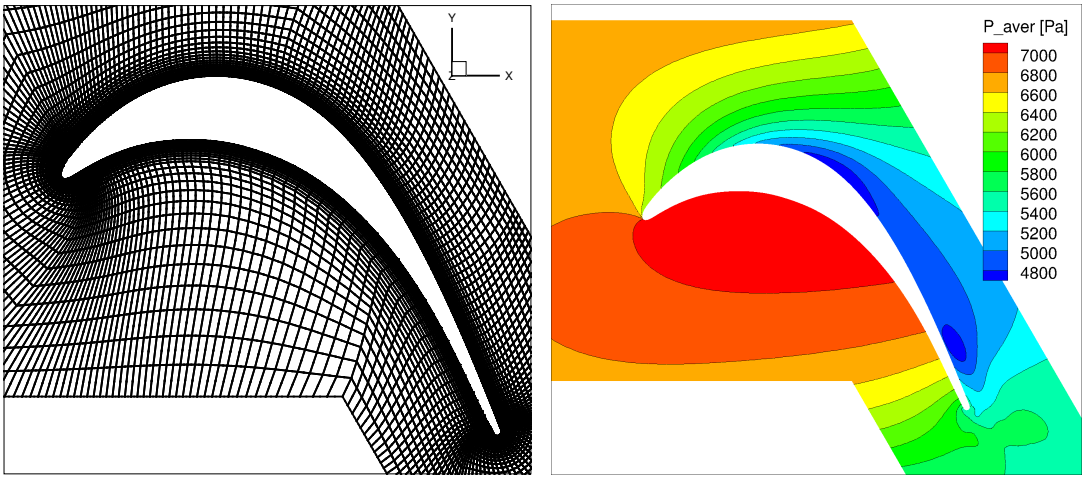
Fig. 3 Strong scalability study with GPU simulations on Summit for a mesh with 15.6M hex elements.

B. Transitional Flow Over the T106C Low Pressure Turbine Cascade

This case setup is taken from the 4th International Workshop on High-Order CFD Methods [37]. The blade has a chord of C = 0.093 m, a pitch to chord ratio of 0.95, and a span to chord ratio of 10%. The inlet condition for the cascade is chosen such that the isentropic exit Mach number is $M_{is} = 0.65$ and the Reynolds number is $Re = 80,000$ based on the isentropic exit velocity. The inlet flow angle for this case is 32.7^o . As the inlet turbulence is very low, the flow has a laminar separation and a relatively slow natural transition.

A direct numerical simulation (DNS) of this case is established in Ref. [38] with an extensive hp-refinement study. The solver used in this reference [38] is the CPU version of the same solver of the current work, namely, the hpMusic solver. In addition, the fine mesh used in this reference was made available for the current GPU work. This test case is used to evaluate the effect of the parameters 'time-eps', 'linear-eps' and Krylov space size, as well as to compare the turnaround time of simulations with three different time solvers: the LU-SGS solver [8, 34, 35] on CPUs, the matrix-free GMRES on GPUs and the explicit RK3 scheme [36] on GPUs.

A closeup view of the mesh around the blade is shown in Fig. 4. It has a total of 148,704 hexahedral elements.



(a) Close view of the mesh. (b) Average pressure contours.

Fig. 4 Mesh provided by Alhawwary et al. [38] and time-averaged solution contours of the p2 simulation with GPU.

Downloaded by Politecnico di Milano on February 4, 2026 | http://arc.aiaa.org | DOI: 10.2514/6.2021-1837

Periodic boundaries are used in the spanwise and pitchwise directions. For a p2 polynomial order, the total number of degrees of freedom is 4 millions/equation. We use the same definition of characteristic time as Ref. [38], $T_c = C/U_\infty$, with C being the blade cord and U_∞ the free stream velocity based on the isentropic exit conditions. Following the analysis in Ref. [38], all simulations are run for $180T_c$, with the last $100T_c$ being used for time averaging. Fig. 4 also shows the average pressure contours around the blade. The average values for the equivalent y+, x+ and z+ for all simulations are 0.5, 10 and 11 respectively.

The first test is related to the physical time step. The GMRES method is tested with different time steps, from a factor of 1 to 100 times the maximum stable explicit RK3 time step. The performance results are shown as the speedup relative to the GPU RK3 simulation when comparing the cost to run one characteristic time T_c. Fig. 5 shows the results for p1, p3 and p5 simulations using the 'linear-eps' as 0.2, the Krylov subspace size as 10 and the 'time-eps' as 0.01.

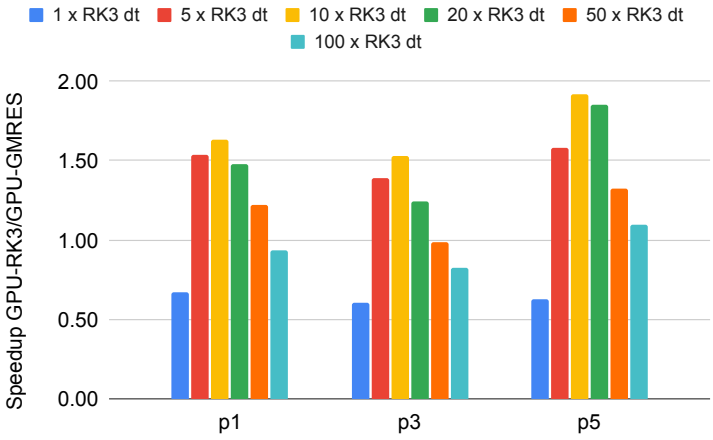


Fig. 5 Results of GPU GMRES speedup relative to the GPU RK3 for running one characteristic time T_c using different physical time steps.

The results in Fig. 5 indicate that there is an optimum time step to be used, and for this case it is around 10 to 15 times the time step from the RK3 stability limit. For a bigger time step, the GMRES solver convergence rate starts to deteriorate as the Jacobian matrix condition number increases. For this case, the calculated improvement by using GPU-GMRES in the turnaround time of the simulations when compared to the GPU-RK3 is about 1.5 for p1 and p3 and 2.0 for p5.

The second study is related to the linear system convergence tolerance parameter. Simulations are run varying the 'linear-eps' between 0.2 and 0.01. In order for the GMRES solver to converge with 0.01 as convergence tolerance, a Krylov subspace size of 30 is used in these tests. The p2 results with a time step 10 times bigger than the RK3 time step and 'time-eps' of 0.01 are shown in Fig. 6.

It is noticeable that for this case the final speedup value does not change significantly varying the 'linear-eps', at least when there are enough Krylov vectors to converge. Moreover, the unsteady residual tolerance of 0.01 is relatively high when compared to the convergence tolerance for steady problems. Since GPU memory is a scarce resource, the 'linear-eps' of 0.2 is chosen with a Krylov subspace of 10, which is already enough to converge in this problem.

Considering the results of the parametric tests, two GPU GMRES simulations are run for the duration of 180 T_c with 'linear-eps' equal to 0.2 and 10 Krylov vectors. One converges the unsteady residual tolerance to 0.01 and the other to 0.001. The complete settings are shown in Table 2, including the one for the CPU LU-SGS run and the GPU RK3 simulation. The speedup when using the GPU GMRES algorithm over the GPU RK3 is a factor of 1.7 and 1.2 for unsteady residual tolerances of 0.01 and 0.001 respectively. For this test case and mesh, the ratio between the maximum and minimum cell Jacobian is 10^4, not so dramatic as the case in the next section.

In order to understand the effect of the parameter 'time-eps' in the solution accuracy, a comparison is made in terms of time- and spanwise-averaged coefficients. Four simulations are considered: CPU, GPU-RK3, GPU-GMRES-eps2 and GPU-GMRES-eps3. The one named 'GPU-GMRES-eps2' has 'time-eps' equal to 0.01 and 'GPU-GMRES-eps3' has 'time-eps' equal to 0.001.

Fig. 7 shows the results for time- and spanwise-averaged pressure coefficient C_p. The CPU and GPU-RK3 curves

Downloaded by Politecnico di Milano on February 4, 2026 | http://arc.aiaa.org | DOI: 10.2514/6.2021-1837

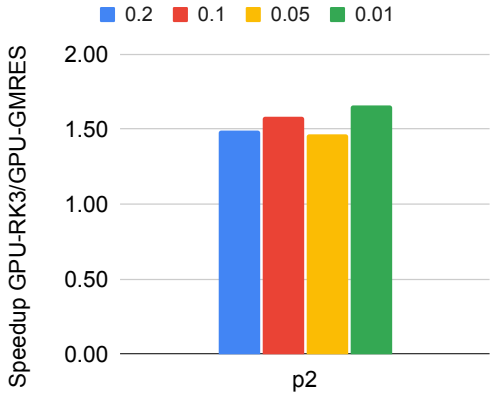


Fig. 6 Results of GPU GMRES speedup relative to the GPU RK3 for running one characteristic time T_c using different GMRES linear convergence tolerance.

Table 2 CPU and GPU simulation final settings for the T106C test case.

Simulation type	CPU	GPU-RK3	GPU-GMRES-eps2	GPU-GMRES-eps3
# Nodes	20	12	12	12
Node specification	Intel Xeon E5-2680 v3 (24 CPU cores)	2 V100 GPUs	2 V100 GPUs	2 V100 GPUs
Wall time (hours)	53	14.5	8.6	12.1
Time solver	LU-SGS	Explicit	GMRES	GMRES
Time scheme	BDF2	RK3	BDF2	BDF2
Unsteady convergence tolerance	0.01	-	0.01	0.001
Time step size	$2e^{-6}$	$1.25e^{-7}$	$1.25e^{-6}$	$1.25e^{-6}$

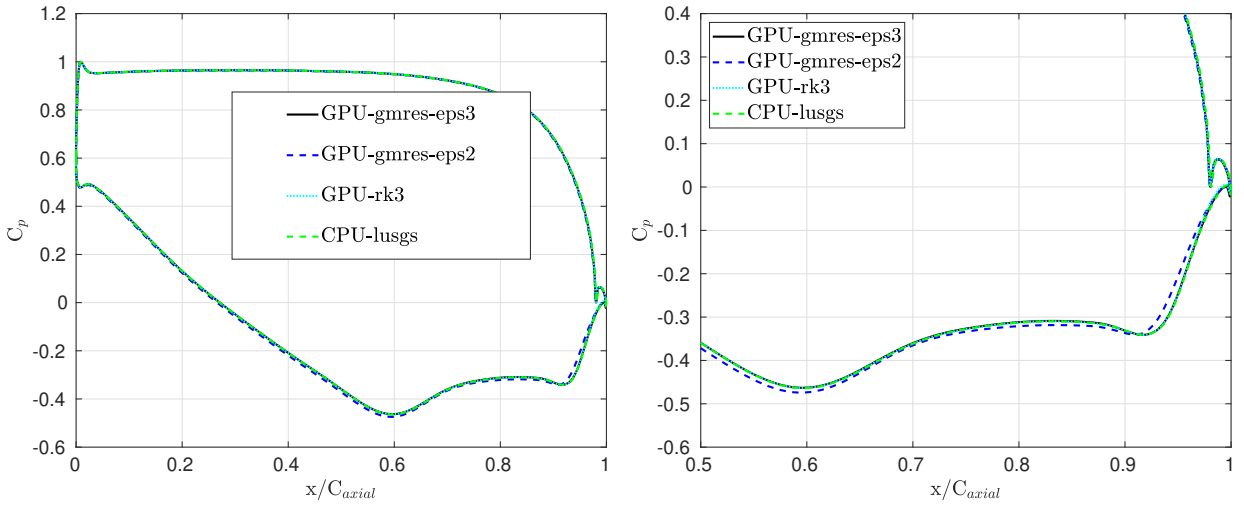
are visually on top of each other, which helps to verify the GPU implementation. When the GMRES time solver is used, the run with 'time-eps' equal to 0.001 is also on top of the explicit time run and the CPU simulation.

Some differences start to appear in the suction side of the blade, close to the trailing edge, when the unsteady convergence tolerance for the GMRES is just 0.01. It is at this location that the laminar flow separation happens with vortex breakdown and transition into turbulent structures. This region is difficult to accurately resolve. The closer view at the trailing edge better shows the difference between the results. It is interesting that at least for this case the GMRES solver needs to have a 'time-eps' equal to 0.001 to match the results of the the LU-SGS solver with a 'time-eps' of 0.01.

Another comparison is made for the time- and spanwise-averaged streamwise friction coefficient C_{fs} in Fig. 8. This coefficient is harder to converge and harder to reach a statistically steady state [38]. The CPU, GPU-RK3 and GPU-GMRES-eps3 hardly show any visual difference on the global view figure. Similar to the pressure coefficient, the blade section where the difference is large for the GPU-GMRES-eps2 is in the suction side, close to the trailing edge as shown in the enlarged view. The GPU-GMRES-eps2 slightly under-predicts the C_{fs} peak at 0.95 of the axial.

The mean lift coefficient $\overline{C_l}$ and drag coefficient $\overline{C_d}$ are also calculated and compared between each other in Table 3. The relative error with respect to the CPU simulation is calculated and also shown in the table. Note that even the GPU-RK3 and CPU simulations have a difference in 0.1% for the $\overline{C_l}$ and 4 drag counts for the $\overline{C_d}$. Possible explanations are the differences in force calculation sampling frequency, time step and time scheme. The GPU-GMRES-eps3 simulation has very similar results as the GPU-RK3 case as well, with a difference of 2 drag counts . Interestingly, the GPU-GMRES-eps2 run has an error of only 5 drag counts when compared with the CPU run, but the $\overline{C_l}$ relative error

Downloaded by Politecnico di Milano on February 4, 2026 | http://arc.aiaa.org | DOI: 10.2514/6.2021-1837



(a) Global view.

(b) Enlarged view near the trailing edge.

Fig. 7 Comparison of the time- and spanwise-averaged pressure coefficient between the CPU simulation and different GPU simulations.

Table 3 Comparison of mean lift and drag coefficients between the CPU simulation and different GPU simulations. Relative error is calculated with respect to the CPU simulation.

Configuration	$\overline{C_l}$	$(\overline{C_l})_{error}$%	$\overline{C_d}$	$(\overline{C_d})_{error}$%
CPU	0.4205	-	1.1402	-
GPU-RK3	0.4201	0.10	1.1398	0.04
GPU-GMRES-eps3	0.4200	0.12	1.1400	0.02
GPU-GMRES-eps2	0.4251	1.09	1.1407	0.04

jumps to 1.1%.

A final comparison is made in Fig. 9 with respect to the power spectral density (PSD) of the pressure at a point in the wake. Following reference [38], the point is located close to the trailing edge with coordinates $(0.8591, -0.5137)C$. The same tool [39] is used in the PSD calculation, with more details provided in Ref. [38]. We conduct a spanwise average using four locations in the spanwise direction, $Z/C \in (0.0, 0.025, 0.05, 0.075)$. The Strouhal number $St = fC/U_\infty$ is calculated based on the chord C, the exit velocity U_∞ and frequency f in Hz. Due to differences in the sampling frequency, the Strouhal number cutoff is different between the GPU-GMRES simulations and the CPU run.

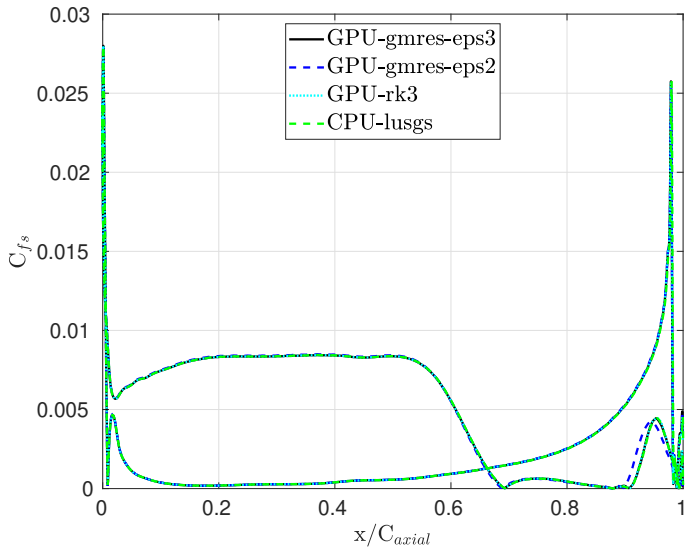
All simulations are able to capture the first fundamental peak and agree in its frequency and magnitude. Moreover, the CPU, GPU-RK3 and GPU-GMRES-eps3 curves seem to be on top of each other. The GPU-GMRES-eps2 appears to capture some of the 2nd and 3rd harmonic frequencies of this first peak, but it slightly damps their magnitudes. The high-frequency slope decay is also slightly damped by the GPU-GMRES-eps2 simulation.

C. Flow Over a High-Lift Configuration of the Common Research Model

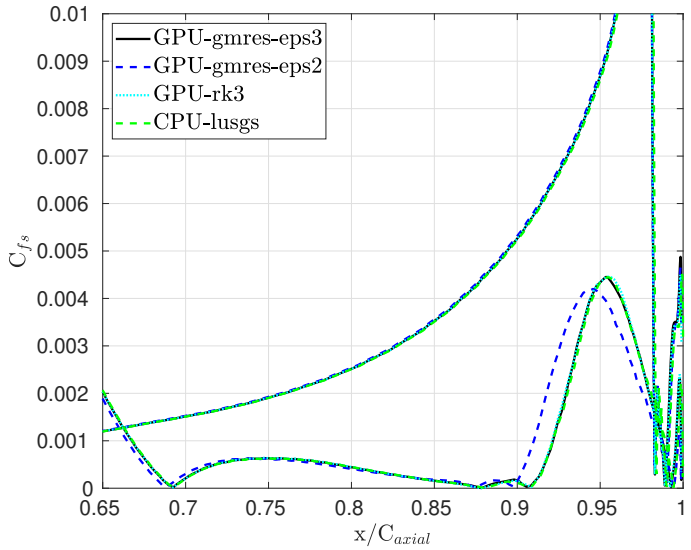
This test case is from the most recent 4th AIAA High-Lift Prediction Workshop (https://hiliftpw.larc.nasa.gov/). Both the geometry and flow conditions are provided on the workshop web site. The High Lift Common Research Model (CRM-HL) is an open-source, publicly-available wing-body high lift configuration being utilized for CFD validation within a broad international CRM-HL ecosystem. Geometry associated with the NASA 10% scale, semi-span model configuration tested in the QinetiQ 5-metre wind tunnel is used for this study. This selected problem is case 2a at the highest angle of attack of 21.46 degrees. The high-order (Q2) computational mesh was generated by Steve Karman of Pointwise. This mesh contains over 4 million mixed elements. Selected views of the computational meshes are displayed in Figure 10. The flow conditions are: free-stream Mach number of 0.2, and Reynolds number based on the

Downloaded by Politecnico di Milano on February 4, 2026 | http://arc.aiaa.org | DOI: 10.2514/6.2021-1837

Downloaded by Politecnico di Milano on February 4, 2026 | http://arc.aiaa.org | DOI: 10.2514/6.2021-1837



(a) Global view.



(b) Enlarged view near the trailing edge.

Fig. 8 Comparison of the time- and spanwise-averaged coefficient of streamwise friction between the CPU simulation and different GPU simulations.

mean aerodynamic chord of 5.49 million. The far-field boundary is placed 1,000-chords away from the geometry. The smallest and largest element sizes differ by more than 6 orders of magnitude.

We report preliminary wall-clock times for both explicit and implicit schemes for the p2 simulations on KU's NVIDIA V100 cluster. For the explicit three-stage SSP Runge-Kutta scheme, the maximum time step for stability is about 1.39e-9s because of some small cells near wall boundaries. The simulation used 30 V100 GPUs, and one time step took roughly 0.053s. For the implicit run, the optimized 2nd-order BDF scheme was used together with a GMRES solver without a preconditioner. We considered two inner convergence tolerance of 0.01 and 0.001. Much bigger time steps of 5e-7s and 1e-6s can be used for the implicit simulation. In fact, for dt = 1.e-6s, we employed an inner tolerance of 0.01. One implicit time-marching step costs 1.1s of wall clock time. Considering that the time-step for the implicit scheme used a much bigger time step, the speedup in this case is roughly 35. When we used an inner tolerance of 0.001, it took roughly twice the wall-clock time for the implicit scheme to converge, resulting in a speed-up of 15.

We are still in the process of running this case, and generating time-converged mean and Reynolds stresses for the

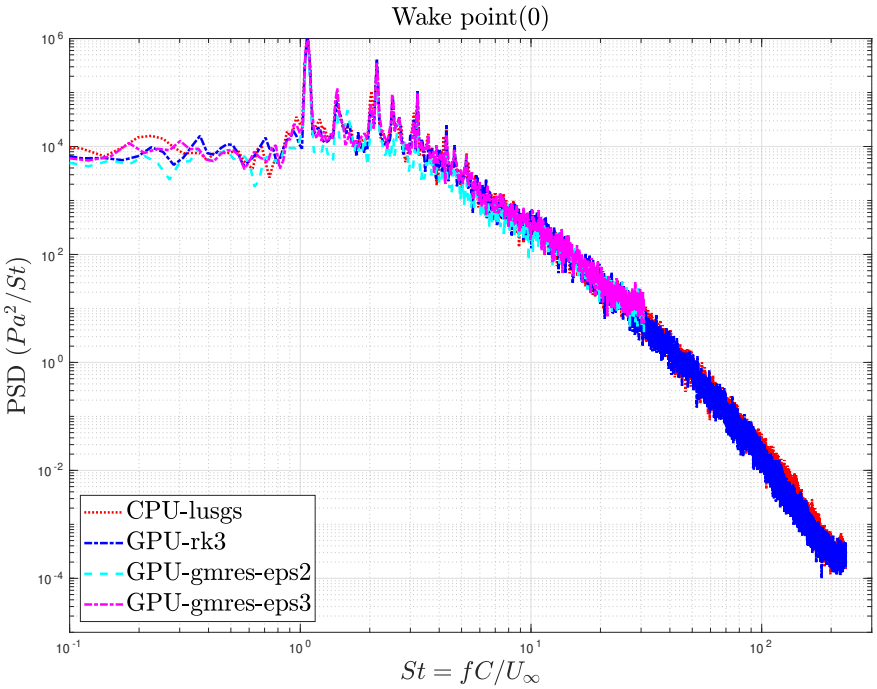


Fig. 9 Comparison of the PSD of pressure at wake point(1).

workshop. But the simulation is proceeding as expected. A preliminary flow field is displayed in Figure 11, which shows the iso-surfaces of the Q-criterion colored by the stream-wise velocity. This picture clearly depicts a vortex-dominated turbulent flow filed.

V. Concluding Remarks

The present paper evaluates the performance of an implementation of the matrix-free GMRES algorithm with the high-order FR/CPR method on GPU clusters for industrial large eddy simulation. Difference in speedups are discussed for four cell types: hexahedrons, prisms, pyramids and tetrahedrons. The effect of increasing the polynomial order from p1 to p5 is also studied. This implicit solver does not need to form the Jacobian matrix and uses less memory, a scarce resource on GPUs, and no matrix preconditioner is considered.

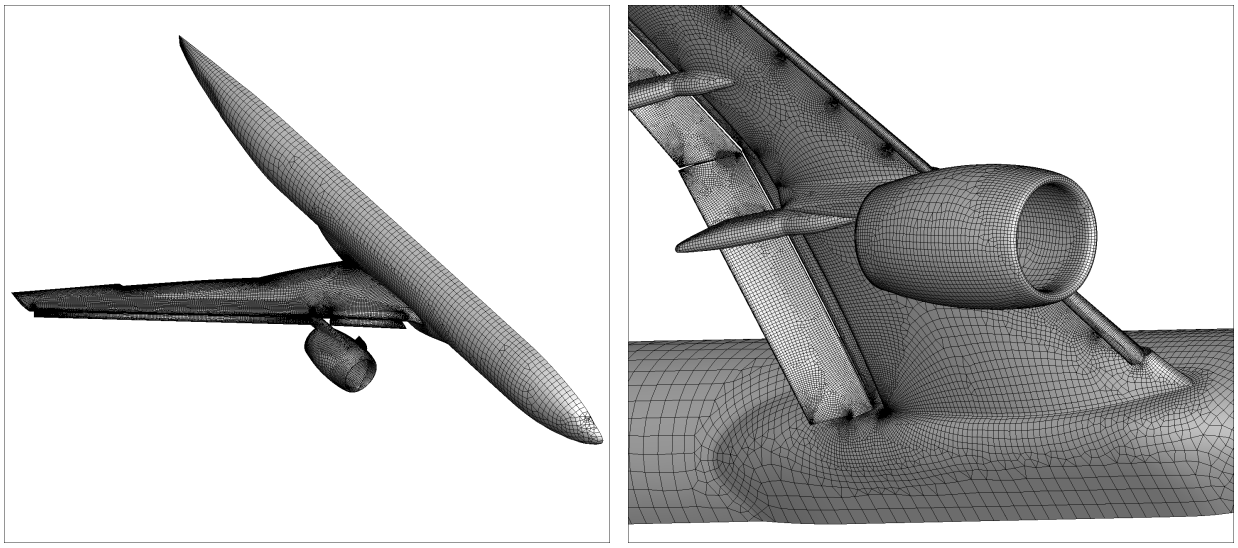
The speedup from one CPU core to a GPU card is measured for NVIDIA V100 GPUs. It is about 380 for hexahedral cells and it grows from 380 to 1300 for tetrahedral cells when increasing the polynomial order from p1 to p5. Depending on the solution order and cell type, the GMRES solver spends around 70% in the residual operator and 15% in the classical Gram-Schmidt algorithm. This last part is important in order keep the implicit solver with similar CPU/GPU speedups and scalability characteristics as the explicit RK3 scheme.

A benchmark case of the flow over the T106c turbine blade is used for parametric tests. A linear convergence tolerance of 0.2 and a Krylov subspace size of 10 seems to give the best performance in this case. Two GMRES simulations are done with different unsteady residual convergence tolerances, and a value of 0.001 seems to be necessary in order to match the explicit simulation results. In this case, the GPU GMRES solver speedup over the GPU RK3 simulation is 1.2.

A test case from the most recent 4th AIAA High-Lift Prediction Workshop is used to measure the performance of this method on mixed meshes and with a higher Reynolds number. Preliminary results on a GPU cluster indicate that the GMRES solver when compared to the RK3 scheme reduces the simulation cost by factor of 15 when the unsteady convergence tolerance is 0.001 and by 35 when the tolerance is 0.01.

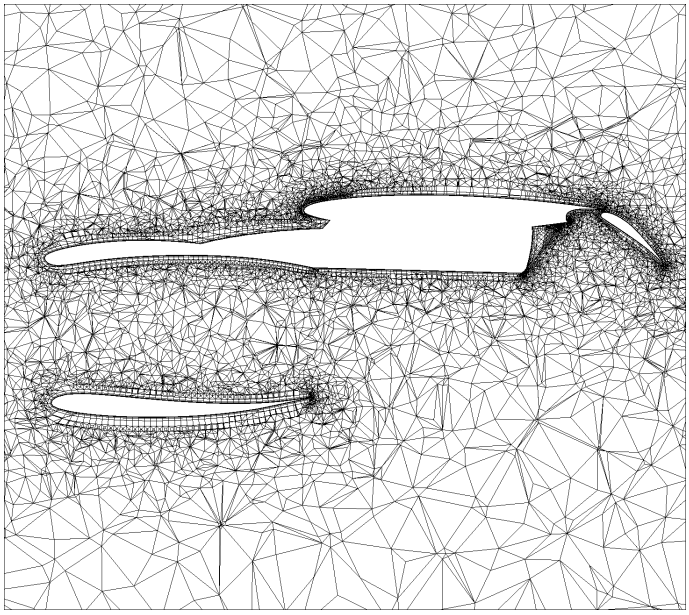
Downloaded by Politecnico di Milano on February 4, 2026 | http://arc.aiaa.org | DOI: 10.2514/6.2021-1837

Downloaded by Politecnico di Milano on February 4, 2026 | http://arc.aiaa.org | DOI: 10.2514/6.2021-1837



(a) Global view.

(b) Enlarged view.



(c) Y-cut view.

Fig. 10 Views of the high-order (Q2) hybrid mesh generated by Pointwise with over 4 million elements.

Acknowledgements

The present research has been supported by GE Global Research and the Army Research Office under Award Number W911NF-20-1-0065. This research used resources of the Oak Ridge Leadership Computing Facility at the Oak Ridge National Laboratory, which is supported by the Office of Science of the U.S. Department of Energy under Contract No. DE-AC05-00OR22725.

References

[1] Wang, Z. J., Fidkowski, K., Abgrall, R., Bassi, F., Caraeni, D., Cary, A., Deconinck, H., Hartmann, R., Hillewaert, K., Huynh, H. T., Kroll, N., May, G., Persson, P., van Leer, B., and Visbal, M., "High-Order CFD Methods: Current Status and Perspective," *International Journal for Numerical Methods in Fluids*, Vol. 72, No. 8, 2013, pp. 811–845.

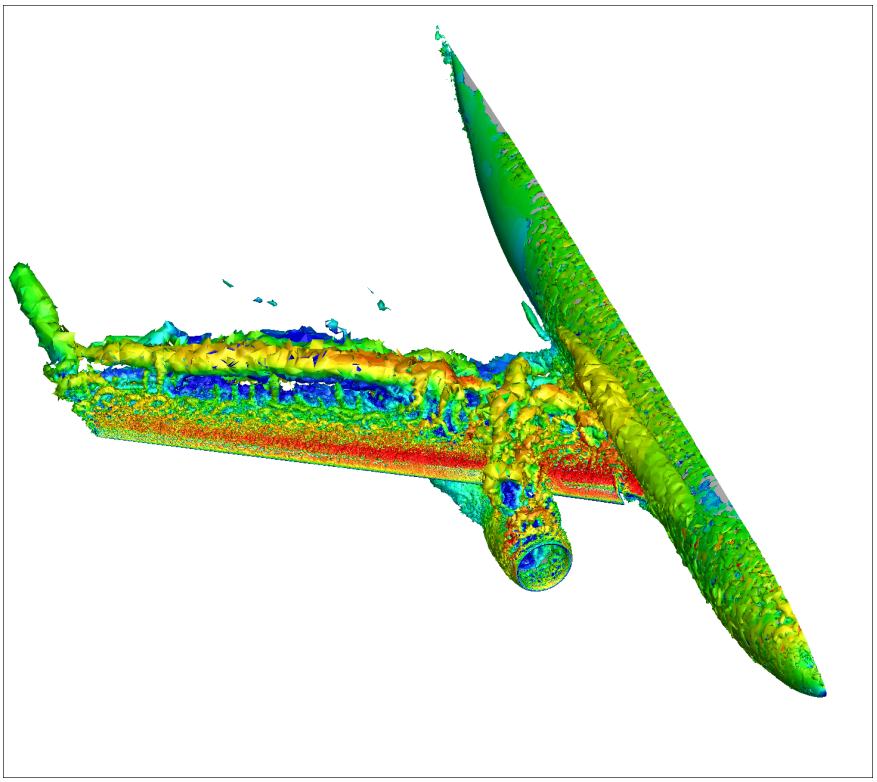


Fig. 11 Instantaneous flow field showing iso-surfaces of the Q-criterion colored by the stream-wise velocity for a p2 simulation.

[2] Slotnick, J., Khodadoust, A., Alonso, J., Darmofal, D., Gropp, W., Lurie, E., and Mavriplis, D., "CFD Vision 2030 study: a path to revolutionary computational aerosciences," NASA CR-2014-218178, NASA, 2014.

[3] Cockburn, B., and Shu, C. W., "TVB Runge-Kutta Local Projection Discontinuous Galerkin Finite Element Method for Conservation Laws V: Multidimensional Systems," *Journal of Computational Physics*, Vol. 141, 1998, pp. 199–224.

[4] Huynh, H. T., "A flux reconstruction approach to high-order schemes including discontinuous Galerkin methods," *18rd AIAA Computational Fluid Dynamics Conference*, Miami, FL, 2007.

[5] Uranga, A., Persson, P.-O., Drela, M., and Peraire, J., "Implicit Large Eddy Simulation of transition to turbulence at low Reynolds numbers using a Discontinuous Galerkin method," *International Journal for Numerical Methods in Engineering*, Vol. 87, No. 1-5, 2011, pp. 232–261. doi:10.1002/nme.3036.

[6] Beck, A. D., Bolemann, T., Flad, D., Frank, H., Gassner, G. J., Hindenlang, F., and Munz, C.-D., "High-order discontinuous Galerkin spectral element methods for transitional and turbulent flow simulations," *International Journal for Numerical Methods in Fluids*, Vol. 76, No. 8, 2014, pp. 522–548. doi:10.1002/fld.3943.

[7] Wang, Z., Li, Y., Jia, F., Laskowski, G. M., Kopriva, J., Paliath, U., and Bhaskaran, R., "Towards industrial large eddy simulation using the FR/CPR method," *Computers & Fluids*, Vol. 156, 2017, pp. 579–589.

[8] Jia, F., Wang, Z., Bhaskaran, R., Paliath, U., and Laskowski, G. M., "Accuracy, efficiency and scalability of explicit and implicit FR/CPR schemes in large eddy simulation," *Computers & Fluids*, Vol. 195, 2019, p. 104316. doi:https://doi.org/10.1016/j.compfluid.2019.104316, URL `http://www.sciencedirect.com/science/article/pii/S0045793019302762`.

[9] Pazner, W., Franco, M., and Persson, P.-O., "High-order wall-resolved large eddy simulation of transonic buffet on the OAT15A airfoil," *2019 AIAA Aerospace Sciences Meeting*, AIAA Paper No. 2019-1152, San Diego, CA, 2019.

[10] Loppi, N. A., Witherden, F. D., Jameson, A., and Vincent, P. E., "A high-order cross-platform incompressible Navier–Stokes solver via artificial compressibility with application to a turbulent jet," *Computer Physics Communications*, Vol. 223, 2018, pp. 193–205.

Downloaded by Politecnico di Milano on February 4, 2026 | http://arc.aiaa.org | DOI: 10.2514/6.2021-1837

Downloaded by Politecnico di Milano on February 4, 2026 | http://arc.aiaa.org | DOI: 10.2514/6.2021-1837

[11] Meuer, H., Strohmaier, E., Dongarra, J., Simon, H., and Meuer, M., "TOP500 high performance computing list." https://www.top500.org/lists/2019/11/, 2019. URL https://www.top500.org/lists/2019/11/.

[12] Vermeire, B., Witherden, F., and Vincent, P., "On the utility of GPU accelerated high-order methods for unsteady flow simulations: A comparison with industry-standard tools," *Journal of Computational Physics*, Vol. 334, 2017, pp. 497–521.

[13] Xu, C., Deng, X., Zhang, L., Rang, J., Wang, G., Jiang, Y., Cao, W., Che, Y., Wang, Y., Wang, Z., Liu, W., and Cheng, X., "Collaborating CPU and GPU for large-scale high-order CFD simulations with complex grids on the TianHe-1A supercomputer," *Journal of Computational Physics*, Vol. 278, 2014, pp. 275–297.

[14] Khajeh-Saeed, A., and Perot], J. B., "Direct numerical simulation of turbulence using GPU accelerated supercomputers," *Journal of Computational Physics*, Vol. 235, 2013, pp. 241 – 257. doi:https://doi.org/10.1016/j.jcp.2012.10.050, URL http://www.sciencedirect.com/science/article/pii/S0021999112006547.

[15] Romero, J., Crabill, J., Watkins, J., Witherden, F., and Jameson, A., "ZEFR: A GPU-accelerated high-order solver for compressible viscous flows using the flux reconstruction method," *Computer Physics Communications*, Vol. 250, 2020, p. 107169. doi:https://doi.org/10.1016/j.cpc.2020.107169.

[16] Zolfaghari, H., Becsek, B., Nestola, M. G., Sawyer, W. B., Krause, R., and Obrist, D., "High-order accurate simulation of incompressible turbulent flows on many parallel GPUs of a hybrid-node supercomputer," *Computer Physics Communications*, Vol. 244, 2019, pp. 132 – 142. doi:https://doi.org/10.1016/j.cpc.2019.06.012, URL http://www.sciencedirect.com/science/article/pii/S0010465519301997.

[17] Oyarzun, G., Chalmoukis, I. A., Leftheriotis, G. A., and Dimas, A. A., "A GPU-based algorithm for efficient LES of high Reynolds number flows in heterogeneous CPU/GPU supercomputers," *Applied Mathematical Modelling*, Vol. 85, 2020, pp. 141 – 156. doi:https://doi.org/10.1016/j.apm.2020.04.010.

[18] Aissa, M., Verstraete, T., and Vuik, C., "Toward a GPU-aware comparison of explicit and implicit CFD simulations on structured meshes," *Computers & Mathematics with Applications*, Vol. 74, No. 1, 2017, pp. 201 – 217. doi:https://doi.org/10.1016/j.camwa.2017.03.003, URL http://www.sciencedirect.com/science/article/pii/S0898122117301438, 5th European Seminar on Computing ESCO 2016.

[19] Jourdan, E., and Wang, Z. J., "Efficient Implementation of the FR/CPR Method on GPU Clusters for Industrial Large Eddy Simulation," *AIAA Aviation 2020 Forum*, Reno, NV, 2020.

[20] Franco, M., Camier, J.-S., Andrej, J., and Pazner, W., "High-order matrix-free incompressible flow solvers with GPU acceleration and low-order refined preconditioners," *Computers & Fluids*, Vol. 203, 2020, p. 104541. doi:https://doi.org/10.1016/j.compfluid.2020.104541.

[21] May, G., and Jameson, A., "Efficient Relaxation Methods for High-Order Discretization of Steady Problems," *Advances in Computational Fluid Dynamics*, Vol. 2, 2011. doi:10.1142/9789814313193_0013.

[22] Knoll, D., and Keyes, D., "Jacobian-free Newton–Krylov methods: a survey of approaches and applications," *Journal of Computational Physics*, Vol. 193, No. 2, 2004, pp. 357 – 397. doi:https://doi.org/10.1016/j.jcp.2003.08.010, URL http://www.sciencedirect.com/science/article/pii/S0021999103004340.

[23] Saad, Y., and Schultz, M. H., "GMRES: A Generalized Minimal Residual Algorithm for Solving Nonsymmetric Linear Systems," *SIAM Journal on Scientific and Statistical Computing*, Vol. 7, No. 3, 1986, pp. 856–869. doi:10.1137/0907058, URL https://doi.org/10.1137/0907058.

[24] Wang, Z. J., and Gao, H., "A unifying lifting collocation penalty formulation including the discontinuous Galerkin, spectral volume/difference methods for conservation laws on mixed grids," *Journal of Computational Physics*, Vol. 2228, No. 21, 2009, pp. 8161–8186.

[25] Huynh, H. T., Wang, Z. J., and Vincent, P. E., "High-order methods for computational fluid dynamics: a brief review of compact differential formulations on unstructured grids," *Computers & Fluids*, Vol. 98, 2014, pp. 209–220.

[26] Wang, Z. J., "A perspective on high-order methods in computational fluid dynamics," *Science China Physics, Mechanics & Astronomy*, Vol. 59, No. 1, 2016, p. 614701.

[27] Vincent, P. E., Castonguay, P., and Jameson, A., "A new class of high-order energy stable flux reconstruction schemes," *Journal of Scientific Computing*, Vol. 47, No. 1, 2011, pp. 50–72.

[28] Park, J. S., You, H., and Kim, C., "Higher-order multi-dimensional limiting process for DG and FR/CPR methods on tetrahedral meshes," *Computers & Fluids*, Vol. 154, 2017, pp. 322–334.

[29] Yu, M., Wang, Z. J., and Liu, Y., "A numerical method for solving incompressible viscous flows problems," *Journal of Computational Physics*, Vol. 259, 2014, pp. 70–95.

[30] Roe, P. L., "Approximatte Riemann Solvers, Parameter Vectors, and Difference Schemes," *Journal of Computational Physics*, Vol. 43, No. 2, 1981, pp. 357–372.

[31] Bassi, F., and Rebay, S., "A high order discontinuous Galerkin method for compressible turbulent flows." *Discontinuous Galerkin Methods. Theory, Computation and Applications*, edited by B. Cockburn, G. Karniadakis, and C. Shu, Springer, 2000, pp. 77–88.

[32] Saad, Y., and van der Vorst, H. A., "Iterative solution of linear systems in the 20th century," *Journal of Computational and Applied Mathematics*, Vol. 123, No. 1, 2000, pp. 1 – 33. doi:https://doi.org/10.1016/S0377-0427(00)00412-X, URL http://www.sciencedirect.com/science/article/pii/S037704270000412X, numerical Analysis 2000. Vol. III: Linear Algebra.

[33] Saad, Y., *Iterative Methods for Sparse Linear Systems*, 2nd ed., Society for Industrial and Applied Mathematics, 2003. doi:10.1137/1.9780898718003, URL https://epubs.siam.org/doi/abs/10.1137/1.9780898718003.

[34] Yoon, S., and Jameson, A., "Lower-upper Symmetric-Gauss-Seidel method for the Euler and Navier-Stokes equations," *AIAA Journal*, Vol. 26, No. 9, 1988, pp. 1025–1026. doi:10.2514/3.10007.

[35] Chen, R. F., and Wang, Z. J., "Fast, block lower-upper symmetric Gauss-Seidel scheme for arbitrary grids." *AIAA Journal*, Vol. 38, No. 12, 2000, pp. 2038, 2245.

[36] Gottlieb, S., and Shu, C. W., "Total variation diminishing Runge-Kutta schemes," *Mathematics of Computation of the American Mathematical Society*, Vol. 67, No. 221, 1998, pp. 73–85.

[37] Vincent, P., Wang, Z. J., Ekaterinaris, J., Huynh, H. T., Kroll, N., and Hillewaert, K., "4th International Workshop on High-Order CFD Methods," https://how4.cenearo.be, 2016. URL https://how4.cenearo.be.

[38] Alhawwary, M., and Wang, Z. J., "On the mesh resolution of industrial LES based on the DNS of flow over the T106C turbine," *Advances in Aerodynamics*, Vol. 1, No. 21, 2019. doi:10.1186/s42774-019-0023-6.

[39] Alhawwary, M., "A C++ toolbox for computing Discrete and Fast Fourier Transforms (DFT,FFT), Power Spectral Density (PSD) estimates, and the sound pressure level (SPL) in (dB)." https://github.com/mhawwary/FFTpsd, 2019. URL https://github.com/mhawwary/FFTpsd.

Downloaded by Politecnico di Milano on February 4, 2026 | http://arc.aiaa.org | DOI: 10.2514/6.2021-1837